

Spring Boot 4 - Spring Security pour une API REST

Table des matières

1. Nathaniel Vaur Henel
2. Spring Security avec Spring Boot 4
    1. Exercice pratique

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**



Nathaniel Vaur Henel

L'auteur

Auteur

👨 Nathaniel Vaur Henel 🤘

- 👨‍🏫 Formateur
- 👨‍💻 Tech leader
- 🧉 Expert Java
- 💻 20 ans d'expérience en développement, principalement en Java
- 📜 Créateur de contenus pour dévs : dev-sensei.digicrafters.fr
- Discord : `@nathanielvaurhenel`



framagit.org/nathvh/me



dev-sensei.digicrafters.fr



linkedin.com/in/nathaniel-vaur-henel

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**



Spring Security avec Spring Boot 4

Introduction au module de sécurité

Spring Security est le framework de référence pour gérer l'authentification et l'autorisation dans l'écosystème Spring.

Dans ce cours, nous nous appuyons sur une application réelle construite avec :

- Spring Boot 4
- Spring Security
- JWT (JSON Web Token)
- Base de données relationnelle
- UserDetailsService personnalisé

Objectif 🎯 : comprendre **chaque brique technique** à travers le code fourni.

> La sécurité n'est pas une option. C'est une architecture.

> Je vous montrerai le code lié à cette partie du projet au fur et à mesure

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Dépendances

Maven

```xml
<dependencies>
  <!-- Security -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <!-- OAuth2 Resource Server (JWT support) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
  </dependency>
</dependencies>
```

Gradle (Groovy DSL) :

```
dependencies {
  implementation 'org.springframework.boot:spring-boot-starter-security'
  implementation 'org.springframework.boot:spring-boot-starter-oauth2-resource-server'
}
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Effet de l’ajout des dépendances

Ajouter **spring-boot-starter-security** suffit pour que :

- Tous les endpoints soient protégés
- Un mot de passe généré apparaisse au démarrage
- Un login HTTP Basic soit activé par défaut

⚠️ Mais sans configuration explicite, la sécurité est :

- Générique
- Non adaptée à une API REST moderne
- Incompatible avec une architecture JWT stateless

> 🧔 Ajouter la dépendance active la sécurité. La config la rend intelligente.

Architecture globale de l'application

L'application est structurée autour des composants suivants :

- Entité utilisateur `AppUser`
- Repository `AppUserRepository`
- Service métier `UserService`
- Implémentation `DbUserDetailsService`
- Configuration `SecurityConfig`
- Service JWT `JwtService`
- Configuration JWT `JwtConfig`
- Contrôleur `AuthController`
- Contrôleur de test `TestController`

Flux général 🔁 :

1. Inscription
2. Authentification
3. Génération du JWT
4. Accès sécurisé aux endpoints

Entité AppUser

L'entité représente un utilisateur stocké en base.

Ce que dit **exactement** le code :

- Table : `users` avec contrainte d'unicité sur `username` (`@UniqueConstraint`).
- Champs :
  - `id` (auto-incrément via `GenerationType.IDENTITY`)
  - `username`
  - `passwordHash` (hash, **jamais** le mot de passe en clair)
  - `role` (un seul rôle par user dans ce projet, ex: `ROLE_USER`)

Responsabilité : représenter un utilisateur persistant.

Bonne pratique 💡 : ne **jamais** exposer directement le `passwordHash` via API (ni en DTO, ni dans les logs).

Repository AppUserRepository

Interface héritant de `JpaRepository`.

Responsabilité : accès aux données utilisateur.

Méthodes clés (générées automatiquement par Spring Data via le nom) :

```
Optional<AppUser> findByUsername(String username);
boolean existsByUsername(String username);
```

- `findByUsername` : utilisé par `DbUserDetailsService` pendant le login.
- `existsByUsername` : utilisé par `UserService` pour garantir l'unicité avant création.

UserService

Couche métier.

Responsabilités :

- Création d'utilisateur
- Encodage du mot de passe
- Logique métier liée aux comptes

Encodage via `PasswordEncoder`.

> On ne stocke **jamais** un mot de passe en clair. **Jamais !**

UserDetailsService personnalisé

Classe : `DbUserDetailsService`

Implémente `UserDetailsService`.

Rôle 🎯 : charger un utilisateur depuis la base pour Spring Security.

```
@Override
public UserDetails loadUserByUsername(String username) {
    AppUser user = repository.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException("User not found"));

    return new User(
        user.getUsername(),
        user.getPassword(),
        authorities
    );
}
```

Spring Security s’appuie exclusivement sur cette méthode.

Configuration SecurityConfig

Classe centrale de configuration (API REST + front JS séparé).

Ce que configure **exactement** le code :

- CORS activé (`.cors(Customizer.withDefaults())`) + une source CORS dédiée
- CSRF désactivé (`AbstractHttpConfigurer::disable`)
- API stateless : pas de session (`SessionCreationPolicy.STATELESS`)
- Autorisations :
  - `OPTIONS /**` autorisé (préflight navigateur)
  - `POST /auth/register` et `POST /auth/login` publics
  - tout le reste → authentifié
- JWT côté Spring Security via **OAuth2 Resource Server** (`.oauth2ResourceServer().jwt(...)`)

Spring Boot 4 utilise uniquement `SecurityFilterChain` (plus de `WebSecurityConfigurerAdapter`).

Configuration CORS

Pourquoi configurer CORS ? ✋

Quand le front (React/Vue/Angular) tourne sur un autre host/port, le navigateur applique la *same-origin policy*.

➡️ Sans CORS, les appels API sont bloqués **par le navigateur** (pas par Spring).

Ce que fait le bean `CorsConfigurationSource` du projet :

- autorise les origins de dev :
  - `http://localhost:3000`
  - `http://localhost:5173`
- autorise les méthodes : `GET, POST, PUT, PATCH, DELETE, OPTIONS`
- autorise les headers : `Authorization`, `Content-Type`
- `allowCredentials(false)` car on n'utilise pas de cookies de session

> 🧔 Adapter le CORS selon votre projet. Il est possible de configurer une Gateway pour ne pas avoir à faire une configuration CORS.

Désactivation du CSRF

Pourquoi désactiver CSRF ? ✋

Dans une architecture REST stateless avec JWT :

- pas de session serveur
- pas de cookie de session

Le CSRF vise principalement les attaques via cookies de session.

Donc dans notre cas : CSRF n'apporte pas de protection utile et complique l'API.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Authentification avec AuthenticationManager

Spring utilise un `AuthenticationManager` (récupéré depuis `AuthenticationConfiguration`).

Processus (dans ce projet) :

1. Le controller envoie username/password à `AuthenticationManager.authenticate(...)`
2. Spring appelle `DbUserDetailsService.loadUserByUsername(username)`
3. Spring compare :
   - mot de passe fourni (en clair) vs `passwordHash` (en base)
   - via `PasswordEncoder` (BCrypt)
4. Si OK → `Authentication` authentifié, utilisé ensuite pour générer le JWT

JWT : principe général

Dans ce projet, on trouve concrètement :

- `iss` (issuer)
- `sub` (subject = username)
- `iat` / `exp`
- `scope` (nos rôles Spring Security)

Avantages :

- stateless
- scalable
- microservices friendly

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

JwtService

Responsable de **générer** le token JWT après un login réussi.

Différence importante : ici on utilise l'API Spring Security JWT (Nimbus), pas `jjwt`.

Ce que fait `generate(Authentication auth)` :

- `auth.getName()` → devient le `subject` (`sub`)
- `auth.getAuthorities()` → concaténé en une chaîne `scope` (séparée par espaces)
- Ajout des claims standards :
  - `iss` (issuer)
  - `iat` (issued at)
  - `exp` (expires at)

Bonne pratique 🔐 : garder `exp` relativement court et prévoir un mécanisme de refresh en production.

JwtConfig

Configuration liée au JWT **et** aux mots de passe.

Ce que contient réellement cette classe :

- Un `PasswordEncoder` BCrypt (hash + vérification)
- Un `JwtEncoder` (SIGNE et produit le JWT)
- Un `JwtDecoder` (VALIDE et décode le JWT)

Dans `SecurityConfig`, on active le *Resource Server JWT*.

➡️ Spring Security a donc besoin d'un `JwtDecoder` pour valider les Bearer tokens.

Et notre endpoint `/auth/login` a besoin d'un `JwtEncoder` pour générer les tokens.

> En production, on préférera souvent des clés asymétriques (RSA/ECDSA) ou une solution OIDC.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Filtres de sécurité

On s'appuie sur **Spring Security OAuth2 Resource Server** :

- Spring lit automatiquement `Authorization: Bearer <token>`
- Il appelle le `JwtDecoder` (bean fourni par `JwtConfig`) pour :
  - vérifier la signature
  - vérifier l'expiration (`exp`)
  - construire un `JwtAuthenticationToken`

Le pipeline contient notamment (conceptuellement) :

- CORS filter
- `BearerTokenAuthenticationFilter` (extrait le Bearer token)
- Authentification JWT (via `JwtDecoder`)
- `AuthorizationFilter` (applique les règles d'accès)

👉 Résultat : sans `JwtDecoder` + `.oauth2ResourceServer().jwt(...)`, aucun endpoint protégé ne passe.

© Nathaniel Vaur Henel pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. CC BY-ND 4.0

AuthController

Contrôleur d'authentification.

Endpoints publics :

- `POST /auth/register` : crée un user en DB
- `POST /auth/login` : vérifie username/password et renvoie un JWT

Le contrôleur définit aussi 3 DTO en `record` (Java 17+) :

- `RegisterRequest(username, password)`
- `LoginRequest(username, password)`
- `TokenResponse(token)`

Login (fidèle au code) :

1. Créer un `UsernamePasswordAuthenticationToken` (candidat)
2. Appeler `AuthenticationManager.authenticate(...)`
3. Récupérer un `Authentication` authentifié
4. Générer le JWT via `jwtService.generate(auth)`
5. Retourner `200 OK` + JSON `{ "token": "..." }`

TestController

Contrôleur de démo : `POST /test/secure`

- Il est protégé car `SecurityConfig` n'autorise en public que `POST /auth/register` et `POST /auth/login`.
- Avec un JWT valide en header → réponse OK.
- Sans token (ou token invalide/expiré) → 401.

Extrait fidèle :

```
@PostMapping("/secure")
public String secureEndpoint() {
  return "youpi (JWT valide ✅)";
}
```

Configuration application.yaml

Le fichier configure 2 zones :

- Spring (datasource, JPA, H2 console)
- App (JWT)

Points importants vus dans le code :

- H2 en mémoire (pratique pour le cours) + mode Postgres + tables en minuscules
- Console H2 activée (utile pour montrer la table `users`)
- JWT :
  - `secret` : Base64 (commentaire : générable via `openssl rand -base64 32`)
  - `issuer` : le générateur du token JWT
  - `exp-minutes` : le temps d'expiration en minute du token

Bonne pratique 🧩 : externaliser `app.jwt.secret` (variable d'env, vault/secret manager) dès qu'on sort de la démo.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Cycle complet d’une requête sécurisée

1. Login
2. JWT généré
3. Appel endpoint protégé
4. Filtre JWT valide
5. SecurityContext alimenté
6. Accès accordé

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Gestion des rôles

Dans ce projet, le rôle est stocké :

- en base dans `AppUser.role` (ex: `ROLE_USER`)
- dans le JWT dans le claim `scope` (ex: `"ROLE_USER"`)

Mapping DB → GrantedAuthority

Pourquoi ce mapping est essentiel ? ✋

Spring Security n'autorise pas "un rôle en String".

Il autorise des **authorities** (`GrantedAuthority`).

Donc on transforme `AppUser.role` en `SimpleGrantedAuthority` .

Extrait typique (ce que fait `DbUserDetailsService`) :

```
new SimpleGrantedAuthority(user.getRole())
```

👉 Si `role` vaut `ROLE_USER` , alors l'authority sera exactement `ROLE_USER` .

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Authorities dans le JWT

Lors du login, `JwtService` prend

- `auth.getAuthorities()`

et les convertit en claim :

- `scope` (chaîne séparée par des espaces)

Exemple :

- `scope = "ROLE_USER ROLE_ADMIN"`

C’est ce claim que Spring relira à chaque requête protégée.

JwtAuthenticationConverter

Pont important : `JwtAuthenticationConverter`

Par défaut, Spring Security applique des conventions sur les scopes/authorities.

Ici, `SecurityConfig` force explicitement :

- claim utilisé : `scope`
- prefix ajouté : **aucun** (on garde `ROLE_...` tel quel)

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Autoriser par rôle dans SecurityConfig

Dans une API, on contrôle l'accès **par endpoint**.

Exemple concret : rendre `/admin/**` accessible uniquement aux admins.

```
.authorizeHttpRequests(auth -> auth
  .requestMatchers(HttpMethod.POST, "/auth/register", "/auth/login").permitAll()
  .requestMatchers("/admin/**").hasAuthority("ROLE_ADMIN")
  .anyRequest().authenticated()
)
```

💡 Pourquoi `hasAuthority("ROLE_ADMIN")` ?

- Nos rôles sont déjà préfixés `ROLE_...`
- C'est la comparaison la plus explicite (pas de magie implicite)

Sécuriser par annotations (niveau méthode)

Quand on veut sécuriser **au plus près du code métier**, on peut ajouter la sécurité “method-level”.

1. Activer :

```
@EnableMethodSecurity
```

2. Protéger :

```
@PreAuthorize("hasAuthority('ROLE_ADMIN')")
public void doAdminStuff() { }
```

⚠️ Important :

- Les règles d’URL restent la première barrière
- Les annotations évitent les oublis quand plusieurs routes appellent le même service

Bonnes pratiques

- DTO dédiés (on a déjà des `record` côté AuthController : bonne base)
- Gestion d'erreurs propre : créer une `Exception` pour le nom d'utilisateur et la mapper en `409 Conflict`
- Durée d'expiration raisonnable + refresh token si besoin
- Rotation des secrets / clés (et invalidation)
- Observabilité : logs + métriques sur auth (sans fuite d'infos)
- Tests :
  - tests unitaires sur `UserService`
  - tests d'intégration sur `/auth/login` + `/test/secure`

Une sécurité non testée est une sécurité imaginaire.

Exemple live



spring-security-api-rest ➡️ https://framagit.org/digicrafters/spring-security-4-template-api-rest

Conclusion

Vous avez vu :

- Architecture Spring Security moderne
- Suppression WebSecurityConfigurerAdapter
- Implémentation UserDetailsService
- Authentification stateless
- Intégration JWT
- Sécurisation d'endpoints

Vous êtes prêts à industrialiser cette base.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Exercice pratique

🏋️ *C'est à votre tour de travailler* 🏋️

> 🧔 Télécharger le projet sur framagit.

Objectif :

- Ajouter un rôle ADMIN
- Protéger un endpoint `/admin`
- Restreindre l'accès aux ADMIN uniquement

Étapes attendues :

1. Ajouter rôle dans AppUser
2. Adapter DbUserDetailsService
3. Configurer SecurityConfig
4. Tester avec JWT

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

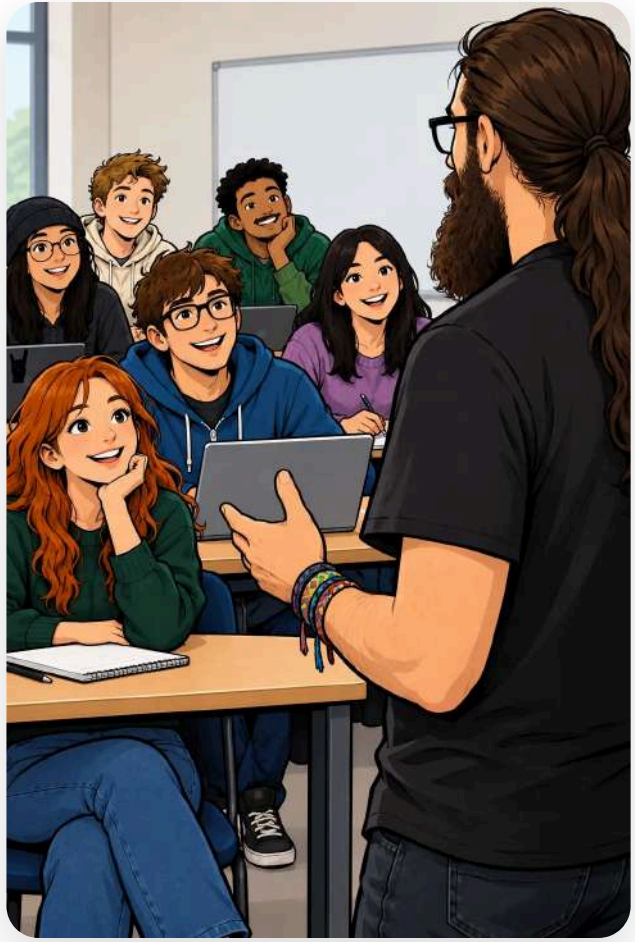
Questions

Spring Security avec Spring Boot 4

Vos questions ✋

Vos demandes d'ajout ✋

- ?



© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

🤝 Merci de ton attention ! 🤝

Reste curieux 🧐 et amuse-toi ! 🎮 🎉

Ce cours t'a été présenté par

 Moi-même, Nathaniel Vaur Henel 🤘

 mon site de partage autour du dév : dev-sensei.digicrafters.fr

 mon Framagit perso, GitLab éthique, libre & français : framagit.org/nathvh/me

 mon GitHub perso, mais inactif : github.com/Nathaniel-Vaur-Henel

 mon LinkedIn, pour suivre mon parcours pro : linkedin.com/in/nathaniel-vaur-henel

 mon Discord pro, pour échanger ou me poser des questions : `@nathanielvaurhenel`